Java Multithreading

Complete Interview Preparation Guide

Theory · Code Examples · Common Pitfalls · Q&A;

Thread Basics	**Thread Lifecycle**	**Runnable vs Thread**
Synchronization	**volatile & atomic**	**wait/notify**
Executor Framework	**Thread Pools**	**Callable & Future**
Locks & Conditions	**Concurrent Collections**	**Common Problems**

Java SE 8+ | Interview Ready | All Levels

1. Thread Fundamentals

1.1 What is a Thread?

A **thread** is the smallest unit of execution within a process. Java supports multithreading natively via the **java.lang.Thread** class and the **java.lang.Runnable** interface. Every Java program has at least one thread — the *main thread* — created automatically by the JVM.

1.2 Ways to Create a Thread

Method 1 – Extend Thread

```
// Extending Thread class
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Thread: " + Thread.currentThread().getName());
    }
}

// Usage
MyThread t = new MyThread();
t.start(); // starts a NEW thread — do NOT call run() directly
```

Method 2 – Implement Runnable (preferred)

```
// Implementing Runnable
class MyTask implements Runnable {
    @Override
    public void run() {
        System.out.println("Running in: " + Thread.currentThread().getName());
    }
}

Thread t = new Thread(new MyTask());
t.start();

// Lambda shorthand (Java 8+)
Thread t2 = new Thread(() -> System.out.println("Lambda thread"));
t2.start();
```

■ *Prefer Runnable/Callable over extending Thread — it keeps your class free to extend something else and separates the task from the thread machinery.*

1.3 Thread Lifecycle

A Java thread passes through the following states (defined in **Thread.State**):

State	Description
NEW	Thread created but start() not yet called
RUNNABLE	Running or ready to run (waiting for CPU)
BLOCKED	Waiting to acquire an intrinsic lock (monitor)
WAITING	Waiting indefinitely (wait(), join(), LockSupport.park())
TIMED_WAITING	Waiting for a specified time (sleep(), wait(ms), join(ms))
TERMINATED	run() has completed or an uncaught exception was thrown

1.4 Key Thread Methods

```
// Common Thread API
Thread t = new Thread(task);
t.start();              // schedule thread for execution
t.join();               // current thread waits until t finishes
t.join(2000);           // wait at most 2 seconds
t.setName("worker-1");
t.setPriority(Thread.MAX_PRIORITY); // 1-10, default 5
t.setDaemon(true);      // JVM exits when only daemon threads remain
t.interrupt();          // request interruption
Thread.sleep(500);      // pause current thread 500 ms (throws InterruptedException)
Thread.currentThread(); // reference to the running thread
```

1.5 Interview Q&A; – Basics

Q: What is the difference between start() and run()?

start() creates a new OS thread and calls run() inside it. Calling run() directly executes it on the CURRENT thread — no new thread is created.

Q: Can we start a thread twice?

No. Calling start() on an already-started thread throws IllegalThreadStateException.

Q: What is a daemon thread?

A background thread (e.g., GC). JVM exits when all non-daemon threads finish, regardless of daemon threads. Set with setDaemon(true) BEFORE start().

2. Synchronization & Thread Safety

2.1 Race Condition & Critical Section

A **race condition** occurs when multiple threads read and write shared data concurrently, producing unpredictable results. The section of code that accesses shared mutable state is called the **critical section**.

Classic race condition example

```
// Counter without sync (BROKEN)
class Counter {
    private int count = 0;
    public void increment() { count++; } // NOT atomic!
    public int get() { return count; }
}
// Two threads calling increment() 1000 times each may give count < 2000
```

2.2 The synchronized Keyword

Java's built-in locking uses **intrinsic locks** (monitors). Every object has one. synchronized can guard a method or an explicit block.

```
// synchronized method & block
class Counter {
    private int count = 0;

    // (a) synchronized instance method — locks 'this'
    public synchronized void increment() { count++; }

    // (b) synchronized block — finer granularity
    public void decrement() {
        synchronized (this) { count--; }
    }

    // (c) synchronized static method — locks the Class object
    public static synchronized void reset() { /* ... */ }
}
```

2.3 volatile Keyword

Declaring a field **volatile** guarantees visibility: every write is flushed to main memory; every read fetches from main memory — no CPU-cache staleness. It does NOT guarantee atomicity for compound operations (check-then-act, i++).

```
// volatile flag pattern
class Worker implements Runnable {
    private volatile boolean running = true;
```

```
    public void stop() { running = false; }

    @Override
    public void run() {
        while (running) {  // always sees the latest value
            doWork();
        }
    }
}
```

2.4 Atomic Classes (java.util.concurrent.atomic)

```
// AtomicInteger example
import java.util.concurrent.atomic.AtomicInteger;

class SafeCounter {
    private AtomicInteger count = new AtomicInteger(0);
    public void increment() { count.incrementAndGet(); }
    public int get()        { return count.get(); }
}
// Internally uses CAS (Compare-And-Swap) — no locking overhead
```

Class	Purpose
AtomicInteger / AtomicLong	Lock-free int/long operations
AtomicBoolean	Lock-free boolean flag
AtomicReference	Lock-free object reference update
AtomicIntegerArray	Lock-free int array
LongAdder	High-contention counter (Java 8+, faster than AtomicLong)

2.5 wait() / notify() / notifyAll()

These Object methods implement the **producer-consumer** pattern using a monitor's *wait set*. They MUST be called from a synchronized context.

```
// Bounded buffer with wait/notify
class BoundedBuffer<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;

    BoundedBuffer(int cap) { this.capacity = cap; }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) wait(); // release lock & sleep
        queue.offer(item);
        notifyAll(); // wake consumers
```

```
        }

        public synchronized T take() throws InterruptedException {
            while (queue.isEmpty()) wait();
            T item = queue.poll();
            notifyAll(); // wake producers
            return item;
        }
    }
```

■ *Always use while (not if) around wait() — guards against spurious wakeups.*

2.6 Interview Q&A; – Synchronization

Q: Difference between synchronized and volatile?

synchronized provides mutual exclusion AND visibility; volatile provides only visibility (no atomicity for compound ops). synchronized is heavier (involves locking) while volatile is lighter.

Q: What is a deadlock? How to prevent it?

Deadlock: two threads each hold a lock the other needs, so both wait forever. Prevention: acquire locks in a consistent global order, use tryLock() with timeout, or prefer higher-level abstractions like java.util.concurrent.

Q: What is livelock?

Threads are NOT blocked but keep reacting to each other and making no progress (e.g., two polite threads each stepping aside for the other indefinitely).

3. Executor Framework & Thread Pools

Managing raw threads is error-prone and inefficient. The **Executor framework** (java.util.concurrent) separates task submission from thread management.

3.1 Executor Hierarchy

```
// Key interfaces
  Executor           // execute(Runnable)
    ExecutorService   // submit(), shutdown(), invokeAll(), invokeAny()
      ScheduledExecutorService  // schedule(), scheduleAtFixedRate()
```

3.2 Executors Factory Methods

Factory Method	Pool Type	Use Case
newFixedThreadPool(n)	Fixed n threads	CPU-bound tasks
newCachedThreadPool()	Grows/shrinks as needed	Many short-lived tasks
newSingleThreadExecutor()	1 thread	Serial task queue
newScheduledThreadPool(n)	Fixed, scheduled	Periodic/delayed tasks
newWorkStealingPool() (J8+)	ForkJoinPool	Parallel streams / divide-conquer

3.3 Callable & Future

```
// Callable example returning a result
  ExecutorService pool = Executors.newFixedThreadPool(4);

  Callable<Integer> task = () -> {
      Thread.sleep(1000);
      return 42;
  };

  Future<Integer> future = pool.submit(task);

  // Do other work here...

  try {
      Integer result = future.get(2, TimeUnit.SECONDS); // blocks until done
      System.out.println("Result: " + result);
  } catch (TimeoutException e) {
      future.cancel(true); // interrupt if running
  } finally {
      pool.shutdown();
  }
```

3.4 CompletableFuture (Java 8+)

```
// Chaining async tasks
CompletableFuture<String> cf = CompletableFuture
    .supplyAsync(() -> fetchData())          // runs in ForkJoinPool
    .thenApply(data -> process(data))        // transform result
    .thenApply(result -> format(result))
    .exceptionally(ex -> "Error: " + ex.getMessage());

cf.thenAccept(System.out::println);          // consume when done

// Combine two futures
CompletableFuture<String> combined = CompletableFuture
    .supplyAsync(() -> "Hello")
    .thenCombine(
        CompletableFuture.supplyAsync(() -> " World"),
        (a, b) -> a + b
    );
System.out.println(combined.get()); // "Hello World"
```

3.5 ThreadPoolExecutor Internals

```
// Custom thread pool
ThreadPoolExecutor pool = new ThreadPoolExecutor(
    2,                              // corePoolSize
    10,                             // maximumPoolSize
    60L, TimeUnit.SECONDS,          // keepAliveTime (idle threads)
    new LinkedBlockingQueue<>(100), // work queue (capacity 100)
    new ThreadFactory() {           // custom factory
        int n = 0;
        public Thread newThread(Runnable r) {
            return new Thread(r, "worker-" + n++);
        }
    },
    new ThreadPoolExecutor.CallerRunsPolicy() // rejection policy
);
```

Rejection Policy	Behavior
AbortPolicy (default)	Throws RejectedExecutionException
CallerRunsPolicy	Caller thread runs the task (back-pressure)
DiscardPolicy	Silently drops the task
DiscardOldestPolicy	Drops the oldest queued task, retries

3.6 Interview Q&A; – Executors

Q: What is the difference between submit() and execute()?

execute(Runnable) fires and forgets — no result, exceptions are swallowed. submit(Callable/Runnable) returns a Future, letting you retrieve the result and handle exceptions via Future.get().

Q: How do you gracefully shut down a pool?

Call shutdown() (no new tasks, existing tasks finish), then optionally awaitTermination(timeout) to wait, then shutdownNow() to interrupt remaining tasks.

4. Locks, Conditions & Concurrent Collections

4.1 ReentrantLock

A more flexible alternative to synchronized: supports tryLock(), timed lock, interruptible lock, and fairness policy.

```
// ReentrantLock usage
import java.util.concurrent.locks.*;

class SafeBank {
    private int balance = 1000;
    private final ReentrantLock lock = new ReentrantLock();

    public void withdraw(int amount) {
        lock.lock();
        try {
            if (balance >= amount) balance -= amount;
        } finally {
            lock.unlock(); // ALWAYS in finally!
        }
    }

    public boolean tryWithdraw(int amount) {
        if (lock.tryLock()) {           // non-blocking attempt
            try { balance -= amount; return true; }
            finally { lock.unlock(); }
        }
        return false;
    }
}
```

4.2 ReadWriteLock

```
// ReentrantReadWriteLock — many readers, one writer
ReadWriteLock rwLock = new ReentrantReadWriteLock();
Lock readLock  = rwLock.readLock();
Lock writeLock = rwLock.writeLock();

// Multiple threads can hold readLock simultaneously
public String getData() {
    readLock.lock();
    try { return data; } finally { readLock.unlock(); }
}

// Only ONE thread at a time can hold writeLock
```

```
    public void setData(String d) {
        writeLock.lock();
        try { data = d; } finally { writeLock.unlock(); }
    }
```

4.3 Condition Variables

```
// Condition replaces wait/notify on a Lock
    Lock lock = new ReentrantLock();
    Condition notEmpty = lock.newCondition();
    Condition notFull  = lock.newCondition();
    Queue<Integer> queue = new LinkedList<>();

    public void put(int v) throws InterruptedException {
        lock.lock();
        try {
            while (queue.size() == CAPACITY) notFull.await();
            queue.add(v);
            notEmpty.signal(); // wake ONE waiting consumer
        } finally { lock.unlock(); }
    }

    public int take() throws InterruptedException {
        lock.lock();
        try {
            while (queue.isEmpty()) notEmpty.await();
            int v = queue.poll();
            notFull.signal();
            return v;
        } finally { lock.unlock(); }
    }
```

4.4 Key Concurrent Collections

Class	Description	Use Case
ConcurrentHashMap	Thread-safe map, segment locking (J8: CAS)	Shared cache / frequency map
CopyOnWriteArrayList	Writes copy the array; reads lock-free	Rarely-written, often-read lists
BlockingQueue (variants)	put/take block on full/empty	Producer-consumer pipelines
LinkedBlockingQueue	Optionally bounded, FIFO	General work queues
ArrayBlockingQueue	Fixed capacity, FIFO	Bounded work queues

PriorityBlockingQueue	Unbounded priority heap	Priority task scheduling
ConcurrentLinkedQueue	Lock-free FIFO queue	Non-blocking message passing

4.5 Synchronization Utilities

```
// CountDownLatch — one-time barrier
CountDownLatch latch = new CountDownLatch(3); // 3 workers

for (int i = 0; i < 3; i++) {
    new Thread(() -> {
        doWork();
        latch.countDown(); // decrement counter
    }).start();
}
latch.await(); // main thread waits until count = 0
System.out.println("All workers done!");
```

```
// CyclicBarrier — reusable rendezvous point
CyclicBarrier barrier = new CyclicBarrier(3, () ->
    System.out.println("All threads reached barrier — next phase!"));

Runnable worker = () -> {
    phase1();
    barrier.await();  // wait for ALL threads
    phase2();
    barrier.await();  // reusable
};
```

```
// Semaphore — limit concurrent access
Semaphore sem = new Semaphore(3); // at most 3 concurrent

void accessResource() throws InterruptedException {
    sem.acquire();      // blocks if 0 permits
    try { useResource(); }
    finally { sem.release(); }
}
```

4.6 Interview Q&A; – Locks & Collections

Q: synchronized vs ReentrantLock — when to choose which?

Use synchronized for simple cases (less boilerplate, automatic unlock). Use ReentrantLock when you need tryLock, timeout, interruptible locking, multiple Conditions, or fairness guarantees.

Q: Why is ConcurrentHashMap faster than Hashtable?

Hashtable locks the entire map on every operation. ConcurrentHashMap (Java 8+) uses CAS + per-bucket synchronization, allowing concurrent reads and mostly concurrent writes with no global lock.

5. Classic Problems & Patterns

5.1 Producer-Consumer (BlockingQueue)

```
// Clean producer-consumer with BlockingQueue
class Producer implements Runnable {
    private final BlockingQueue<Integer> queue;
    Producer(BlockingQueue<Integer> q) { queue = q; }
    @Override public void run() {
        try {
            for (int i = 0; i < 20; i++) {
                queue.put(i);          // blocks if full
                System.out.println("Produced: " + i);
            }
            queue.put(-1);             // poison pill to stop consumer
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
}
class Consumer implements Runnable {
    private final BlockingQueue<Integer> queue;
    Consumer(BlockingQueue<Integer> q) { queue = q; }
    @Override public void run() {
        try {
            while (true) {
                int val = queue.take();   // blocks if empty
                if (val == -1) break;     // poison pill
                System.out.println("Consumed: " + val);
            }
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
}
// Main
BlockingQueue<Integer> q = new LinkedBlockingQueue<>(10);
new Thread(new Producer(q)).start();
new Thread(new Consumer(q)).start();
```

5.2 Singleton (Thread-Safe)

```
// Double-checked locking + volatile (recommended)
public class Singleton {
```

```
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {                    // 1st check (no lock)
            synchronized (Singleton.class) {
                if (instance == null) {            // 2nd check (with lock)
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}

// Alternatively — initialization-on-demand holder (simpler, equally safe)
public class Singleton2 {
    private Singleton2() {}
    private static class Holder {
        static final Singleton2 INSTANCE = new Singleton2();
    }
    public static Singleton2 getInstance() { return Holder.INSTANCE; }
}
```

5.3 Thread-Local Variables

```
// ThreadLocal — per-thread storage
public class UserContext {
    private static final ThreadLocal<String> user = new ThreadLocal<>();

    public static void setUser(String u) { user.set(u); }
    public static String getUser()       { return user.get(); }
    public static void clear()           { user.remove(); } // prevent leak!
}

// Each thread has its own independent copy of 'user'
```

5.4 ForkJoinPool & RecursiveTask

```
// Parallel merge sort skeleton
class SumTask extends RecursiveTask<Long> {
    private final int[] arr;
    private final int start, end;
    private static final int THRESHOLD = 1000;

    @Override
    protected Long compute() {
```

```
            if (end - start <= THRESHOLD) {
                return sequentialSum(arr, start, end);
            }
            int mid = (start + end) / 2;
            SumTask left  = new SumTask(arr, start, mid);
            SumTask right = new SumTask(arr, mid, end);
            left.fork();                    // execute left in parallel
            long rightResult = right.compute();
            long leftResult  = left.join();  // wait for left
            return leftResult + rightResult;
        }
    }
    ForkJoinPool pool = new ForkJoinPool();
    long total = pool.invoke(new SumTask(data, 0, data.length));
```

5.5 Common Pitfalls Summary

Problem	Cause	Fix
Deadlock	Circular lock dependency	Consistent lock ordering / tryLock with timeout
Livelock	Threads react to each other, no progress	Add randomness or back-off
Starvation	Low-priority thread never gets CPU	Fair locks, thread priorities
False Sharing	Threads share a cache line, causing constant invalidation	@Contended or padding
Memory Visibility	Stale cached values read by other threads	volatile, synchronized, Atomic classes
Missed Signal	notify() called before wait()	Always check condition in a loop

5.6 Interview Q&A; – Patterns

Q: What is the difference between CountDownLatch and CyclicBarrier?

CountDownLatch is one-shot: once count reaches zero it cannot be reset. CyclicBarrier is reusable: after all threads arrive, the count automatically resets for the next round.

Q: What is a ThreadLocal leak and how do you avoid it?

If a thread returns to a pool (e.g., Tomcat) without calling ThreadLocal.remove(), the value stays associated with the thread and leaks across requests. Always call remove() in a finally block.

Q: How does volatile ensure visibility but not atomicity?

volatile forces every write to go directly to main memory and every read to come from main memory, so all threads see the latest value. However, count++ is three steps (read, increment, write) and another thread can interleave between them — volatile doesn't prevent that.

6. Quick Reference & Cheat Sheet

6.1 synchronized vs Lock vs volatile vs Atomic

Feature	synchronized	ReentrantLock	volatile	Atomic*
Mutual exclusion	✔	✔	✗	Partial (single op)
Visibility	✔	✔	✔	✔
Reentrancy	✔	✔	N/A	N/A
Try/Timed lock	✗	✔	N/A	N/A
Multiple conditions	✗	✔	N/A	N/A
Performance	Medium	Medium	Low overhead	Low overhead

6.2 Executor Factory Quick Guide

Scenario	Recommended Executor
Fixed number of CPU-intensive tasks	newFixedThreadPool(Runtime.availableProcessors())
Many short-lived I/O tasks	newCachedThreadPool()
Single background worker	newSingleThreadExecutor()
Periodic / delayed tasks	newScheduledThreadPool(n)
Divide-and-conquer / parallel streams	ForkJoinPool.commonPool()
Custom tuning required	new ThreadPoolExecutor(...)

6.3 InterruptedException Best Practice

```
// Handling InterruptedException correctly
    // Option 1: Re-interrupt and return (common in Runnable.run())
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt(); // restore interrupted status
        return;
    }
    // Option 2: Propagate as checked exception (if method signature allows)
```

```
    void doWork() throws InterruptedException {
        Thread.sleep(1000);
    }
    // NEVER silently swallow it — this hides the signal!
    // } catch (InterruptedException e) { } // BAD
```

6.4 Top 15 Interview Questions Recap

1. What is the difference between process and thread?
2. How do you create a thread in Java? (3 ways)
3. Difference between start() and run()?
4. What is thread safety? How to achieve it?
5. Explain volatile. When is it sufficient?
6. What is a deadlock? Give an example and prevention strategy.
7. Explain wait(), notify(), notifyAll() — rules to use them.
8. What is the Executor framework? Why use it?
9. Callable vs Runnable?
10. What is Future and CompletableFuture?
11. Explain ReentrantLock advantages over synchronized.
12. What is ConcurrentHashMap and how is it different from HashMap/Hashtable?
13. What is CountDownLatch vs CyclicBarrier vs Semaphore?
14. What is a ThreadLocal variable? Potential pitfalls?
15. What is the Java Memory Model (JMM) and happens-before relationship?

6.5 Java Memory Model — Happens-Before Rules

The JMM defines when one thread's writes are guaranteed visible to another. Key **happens-before** (HB) relationships:

Rule	Meaning
Program order	Each action HB the next in the same thread
Monitor unlock HB lock	Exiting synchronized HB any subsequent entry
volatile write HB read	Writing a volatile field HB any subsequent read of it
Thread start HB	Thread.start() HB any action in the started thread
Thread join HB	All actions in a thread HB Thread.join() returning
Transitivity	If A HB B and B HB C then A HB C